

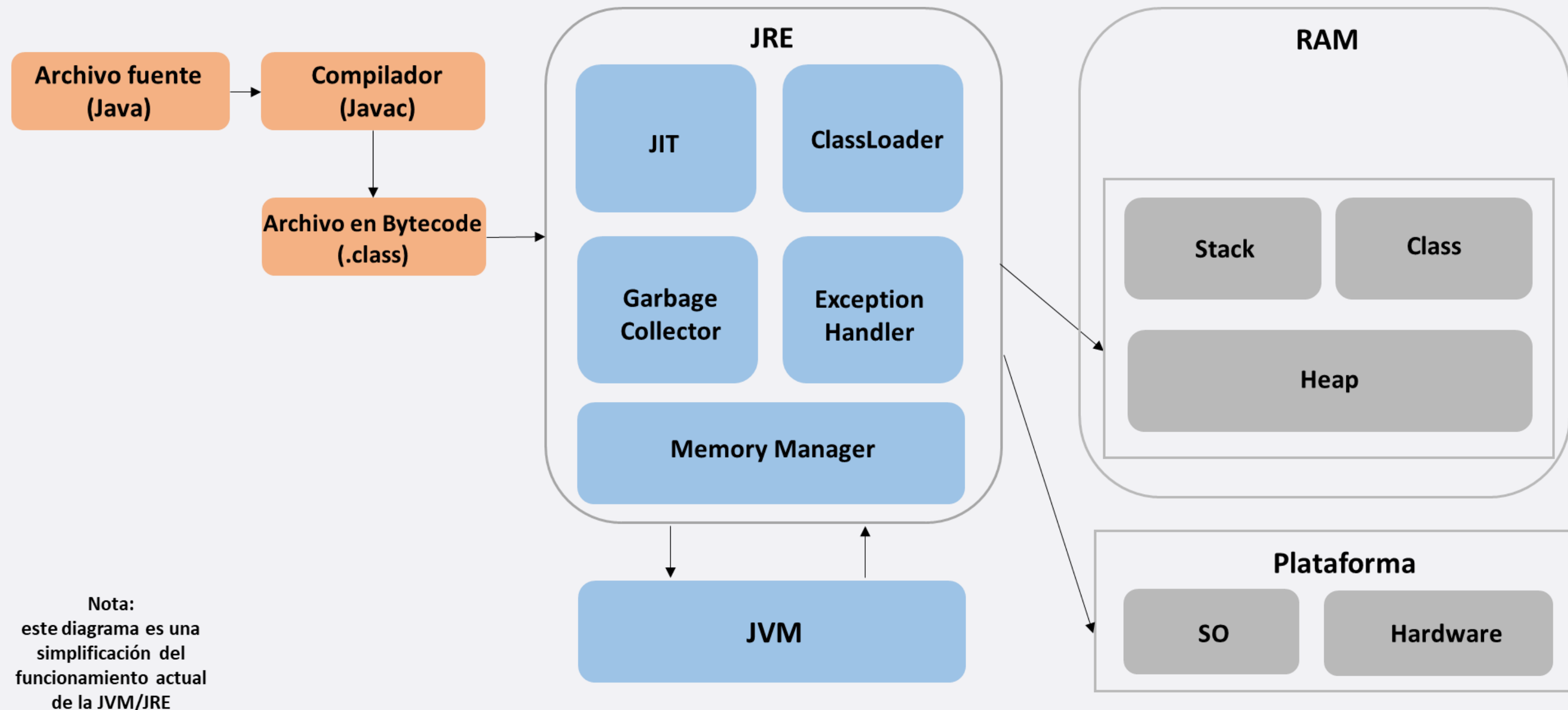
Funcionamiento de la JVM y JRE

Para instalar JAVA se necesita instalar el “kit de desarrollo”. El entorno de desarrollo se denomina JDK (Java Development Kit), que involucra tanto la JVM (Java Virtual Machine) como también el JRE (Java Runtime Enviroment).

- **JRE:** sirve para ejecutar código Java, para correr programas Java.
- **JDK:** tiene muchas herramientas de desarrollo, como ser el compilador, el motor para hacer documentación, etc.
- **JVM:** una máquina virtual corre entre el lenguaje y el SO. Cuando se escribe código Java no se compila a código de máquina. En C cuando se compila se hace en lenguaje de máquina; por ejemplo: Si tengo Linux, debo compilar en C para Linux (lo mismo sucedería en caso de Windows o Mac).

Con una máquina virtual, como es en el caso de Java (o .Net para el caso), cuando se compila, se hace para el lenguaje de la máquina virtual y no para lenguaje de máquina. Lo que se instala para diferentes plataformas es la máquina virtual. Cuando se instala Java, se instala el JRE (y la JVM). El JRE sí se instala para Linux, Windows, Mac. La máquina virtual es lo que cambia en cada plataforma. En cambio, el código se compila una vez en cualquier plataforma y luego puede correr en cualquier máquina virtual de cualquier otra (ejemplo: escribo y compilo con un JDK en Windows, pero puedo correr el programa en un JRE para Linux).

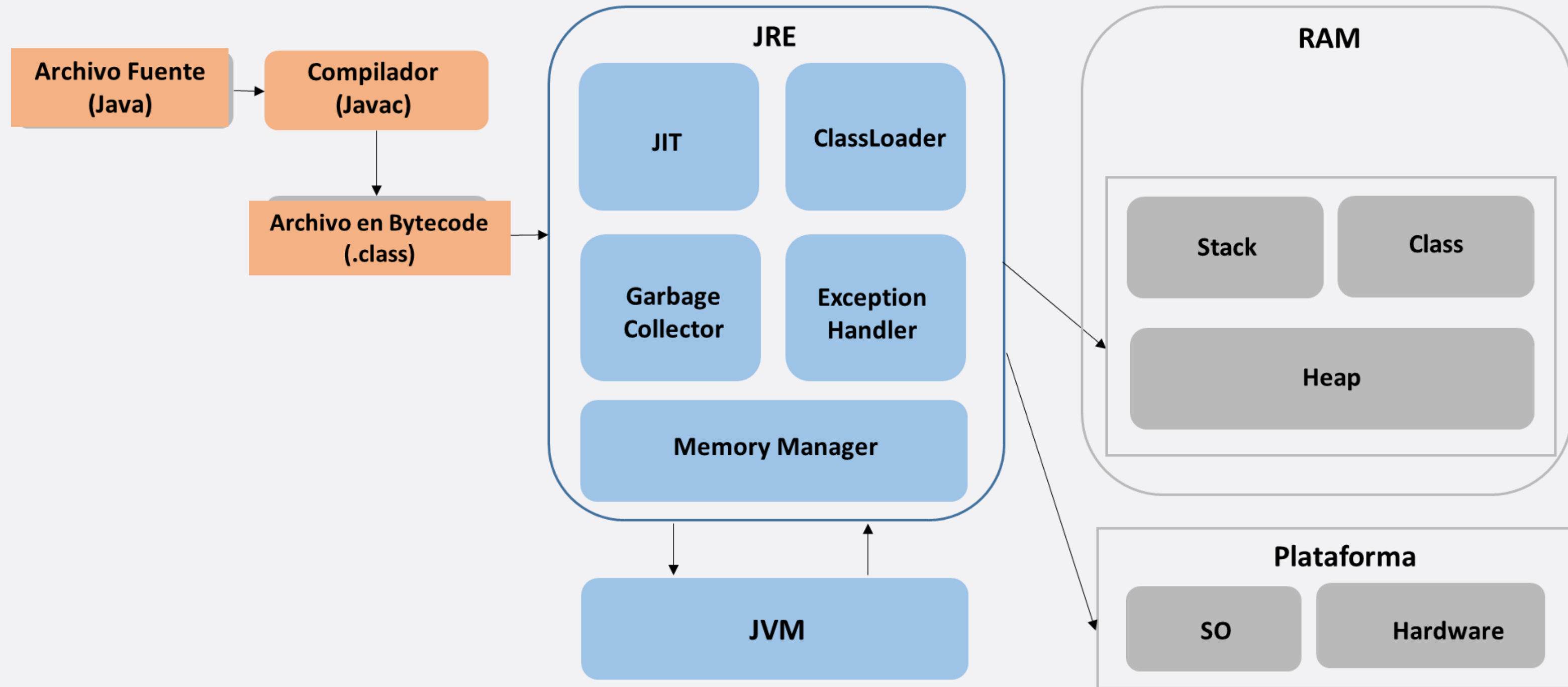
Diagrama de flujo | Primera parte



Procedimiento | Primera parte

- Lo primero que se hace es escribir el archivo de código fuente. El código fuente consiste en instrucciones en lenguaje natural, en inglés.
- El código fuente se debe compilar. El compilador es una de las herramientas que tiene el JDK, con el cual se logra un código en Bytecode. El producto del compilador no es lo que la JVM ejecuta en forma directa. Tampoco es código de máquina.
- El archivo compilado que está en Bytecode es un archivo binario. La JVM no interpreta Bytecode directamente. Hay lenguajes que son interpretados (por ejemplo: PHP, Pearl) y otros que son compilados (por ejemplo: C). Java no es un lenguaje interpretado ni compilado.

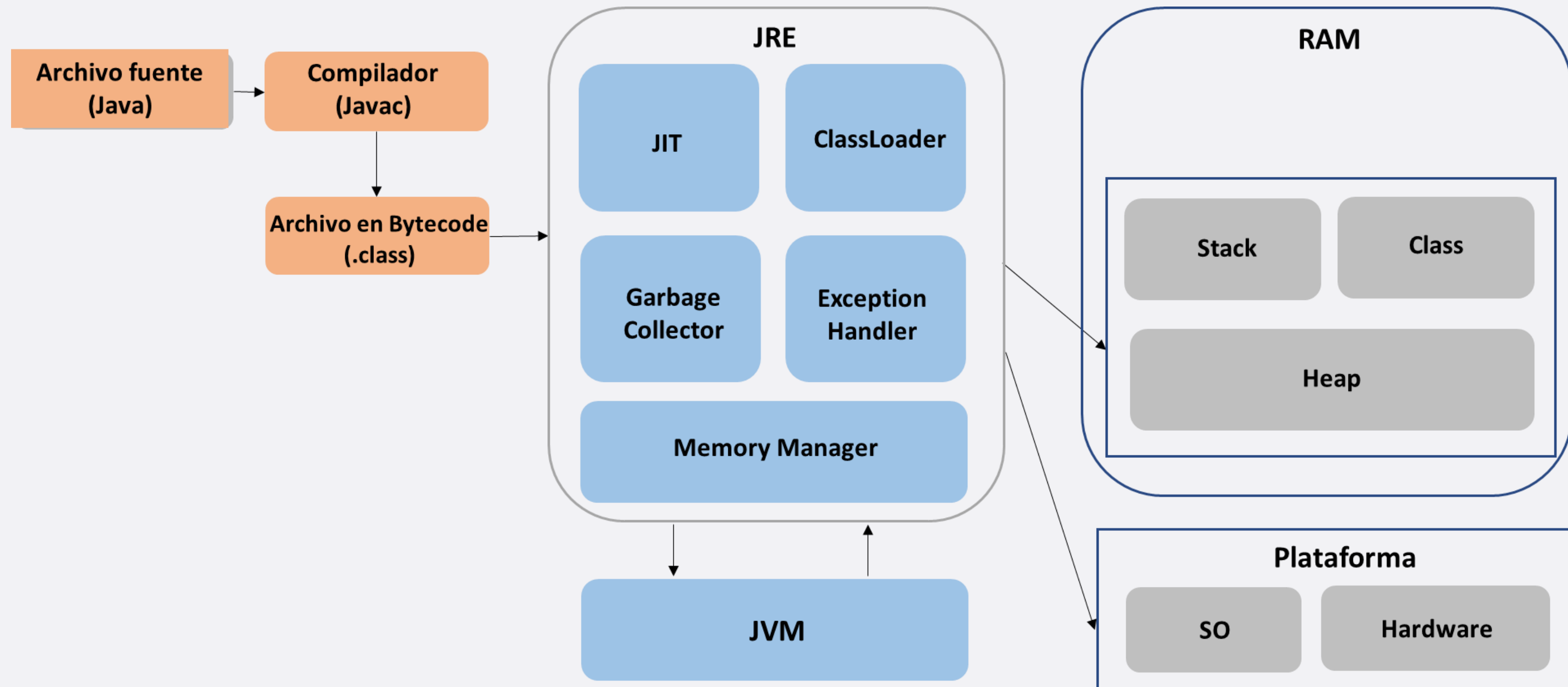
Diagrama de flujo | Segunda parte



Procedimiento | Segunda parte

- Cuando el programa corre, el JRE (Java Runtime Enviroment) toma esos archivos en Bytecode y lo pasa por el JIT (Just In Time compiler), para entonces procesarlo y enviarlo al lenguaje que entiende la JVM (Java Virtual Machine).
- Una vez procesado se utiliza el módulo ClassLoader, el cual levanta a memoria todo lo necesario para poder correr el programa. El ClassLoader levanta a memoria clases, objetos, etc., mediante el módulo de Memory Manager, este módulo habla tanto con la memoria física de la máquina como con la plataforma si es que necesita paginar, etc.
- El Garbage Collector, a diferencia de C, administra la memoria, es decir, cuando una variable o referencia ya no se usa más, lo elimina automáticamente de la memoria. Entonces, al haber una liberación automática de la memoria no hay que hacer una reserva previa (en Java no necesitamos malloc ni tampoco necesitamos liberar la memoria una vez utilizada).
- Por último, está el Exception Handler, es un módulo que lo que hace es manejar los posibles errores que puedan llegar a lanzarse durante la ejecución del programa.

Diagrama de Flujo | Tercera parte



Procedimiento | Tercera parte

- El ClassLoader usa el Memory Manager para acceder a la memoria RAM de la computadora. JAVA es un lenguaje que utiliza una Virtual Machine. Es esta la que usa la memoria física, pero nuestro programa no usa memoria física, sino que usa un área de memoria que la VM administra.
- La JVM reserva una cantidad física de memoria para poder funcionar. Esa memoria está dividida en 3 partes fundamentales: 1. HEAP, como la palabra lo dice, es el “montón” de objetos y demás datos que están ejecutándose en un momento determinado; 2. el área de STACK que maneja la pila de llamadas, es decir, cuando una función llama a otra y le pasa argumentos (y llama a otra que le pasa otros argumentos y así sucesivamente), no solo se va almacenando qué función llama a qué otra función, sino también los argumentos que se van pasando entre funciones y los valores retornados por cada una; 3. el área de CLASS, que esto maneja los valores de variables que son “de Clase” o también llamados “estáticos”.
- El JRE junto con la JVM interactúan con el disco rígido (u otros recursos) o con el SO con la plataforma. El programa nunca interactuará los recursos directamente, siempre mediante VM.

Classpath

- Es el espacio definido en el SO, accedido por el JRE donde se encontrarán todas las librerías del lenguaje para que pueda correr la JVM, funcionar el JRE y ejecutarse nuestros programas. Cualquier librería de utilidades que necesitamos agregar a nuestras aplicaciones (frameworks web, utilidades de audio, librerías de estadísticas, frameworks de persistencia, controladores de bases de datos, etc.).
- CLASSPATH es una variable de entorno del SO que tenemos que establecer para correr nuestros programas Java. Por defecto, el valor de esta variable es el “directorio actual”. Por lo tanto, podríamos hacerla apuntar a un directorio donde tenemos todas las librerías y listo. Normalmente esto no es muy práctico y existen un montón de herramientas para solucionar este problema. Los IDE, por lo general, manejan el CLASSPATH por nosotros.



© **Universidad de Palermo**

Prohibida la reproducción total o parcial de imágenes y textos.